

Microservices & Kafka – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 14 Jan 2027 – 25 Jan 2027 (Days 151–162)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Microservices architecture, Kafka internals, event-driven patterns, API protocols, and 80 LPA interview preparation.

Target Level: Google L5, Meta E4, Amazon SDE3, Microsoft L63+ (80+ LPA TC in India)

PREREQUISITES – Complete FIRST

Order	PDF
1	<code>11-distributed-systems-complete.pdf</code> (CAP, consensus, partitioning)
2	<code>10-system-design-complete.pdf</code> (monolith vs microservices intro)
3	<code>07-auth-devops-complete.pdf</code> (Docker basics)

When: Month 6, Days 156–162 (after distributed systems).

Table of Contents

1. Microservices vs Monolith
2. Service Discovery & API Gateway

3. [Resilience Patterns](#)
4. [Apache Kafka Deep Dive](#)
5. [Message Queue Comparison](#)
6. [Event-Driven Architecture](#)
7. [CQRS & Event Sourcing](#)
8. [gRPC vs REST vs GraphQL](#)
9. [Interview Q&A \(15 Questions\)](#)
10. [Where to Practice](#)

1. Microservices vs Monolith

Monolith Architecture



Advantages:

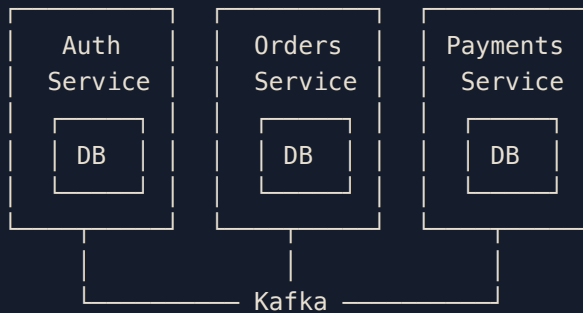
- Simple deployment (one artifact)
- Easy local development and debugging
- ACID transactions across modules
- No network latency between components
- Straightforward testing

Disadvantages:

- Scaling requires scaling everything together
- Technology lock-in (one language/framework)
- Large codebase becomes unmaintainable
- Single point of failure

- Slow deployment cycles (must deploy entire app)
- Team coordination overhead as org grows

Microservices Architecture



Advantages:

- Independent deployment and scaling per service
- Technology diversity (Node.js for API, Python for ML, Go for workers)
- Team autonomy (Conway's Law alignment)
- Fault isolation (payment failure doesn't crash auth)
- Faster iteration for individual teams

Disadvantages:

- Distributed system complexity (network failures, latency)
- Data consistency challenges (no cross-service ACID)
- Operational overhead (monitoring, deployment, service mesh)
- Testing complexity (integration tests across services)
- Debugging across service boundaries
- Eventual consistency by default

When to Choose What

Factor	Monolith	Microservices
Team size	< 10 engineers	10+ engineers, multiple teams
Domain complexity	Simple, well-understood	Complex, multiple bounded contexts
Scale requirements	Moderate	High, uneven per component
Deployment frequency	Weekly/monthly	Multiple times per day
Organization maturity	Startup, early product	Established DevOps/SRE practices

The Modular Monolith: A pragmatic middle ground — well-bounded modules within a single deployable, with clear interfaces. Extract to microservices only when scaling or team boundaries demand it. Shopify and Basecamp advocate this approach.

Migration Strategy: Strangler Fig Pattern

```
Phase 1: Monolith handles all traffic
Phase 2: New features built as microservices, routed via API gateway
Phase 3: Gradually extract existing modules (auth first, then payments)
Phase 4: Monolith becomes thin shell or is fully replaced
```

2. Service Discovery & API Gateway

Service Discovery

Problem: In microservices, service instances are ephemeral (auto-scaling, container restarts). Clients can't hardcode IP addresses.

Client-Side Discovery:

```
Client → Service Registry (Consul/Eureka) → Get instance list → Load balance → Call service
```

- Client responsible for discovery and load balancing
- Used by: Netflix Eureka + Ribbon, gRPC with Consul

Server-Side Discovery:

Client → Load Balancer → Service Registry → Route to healthy instance

- Infrastructure handles discovery transparently
- Used by: Kubernetes Services, AWS ALB + ECS/EKS

Service Registry Options:

Tool	Type	Used By
Consul	Distributed, health checks	HashiCorp stack
etcd	Key-value, strong consistency	Kubernetes
Eureka	AP, self-registration	Netflix (legacy)
Kubernetes DNS	Built-in K8s service discovery	K8s native apps

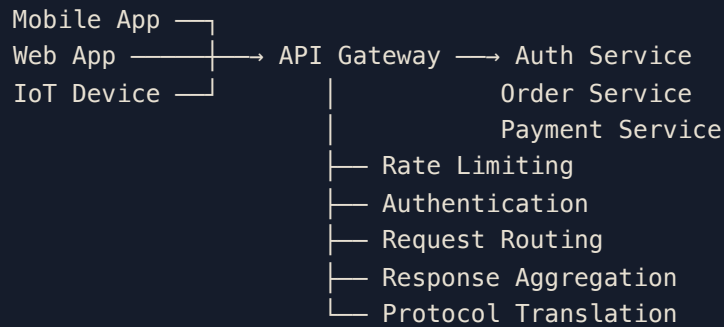
Health Checks:

```
// HTTP health check endpoint
app.get('/health', async (req, res) => {
  const dbHealthy = await checkDatabase();
  const cacheHealthy = await checkRedis();

  if (dbHealthy && cacheHealthy) {
    res.status(200).json({ status: 'healthy' });
  } else {
    res.status(503).json({ status: 'unhealthy', details: { db: dbHealthy, cache: c
  }
});
```

API Gateway

Purpose: Single entry point for all client requests. Routes to appropriate microservices.



Responsibilities:

1. **Routing:** Map `/api/orders` → Order Service
2. **Authentication/Authorization:** Validate JWT, check permissions
3. **Rate Limiting:** Prevent abuse (token bucket, sliding window)
4. **Request/Response Transformation:** gRPC ↔ REST conversion
5. **Aggregation:** Combine responses from multiple services (BFF pattern)
6. **Caching:** Cache frequent responses at the edge
7. **Circuit Breaking:** Fail fast when downstream services are down

API Gateway Options:

Gateway	Type	Best For
Kong	Open source, plugin ecosystem	General purpose
AWS API Gateway	Managed, serverless	AWS-native apps
NGINX/Envoy	High performance proxy	Service mesh data plane
GraphQL Gateway (Apollo)	Schema stitching	GraphQL federation

Backend for Frontend (BFF): Instead of one gateway for all clients, create specialized gateways:

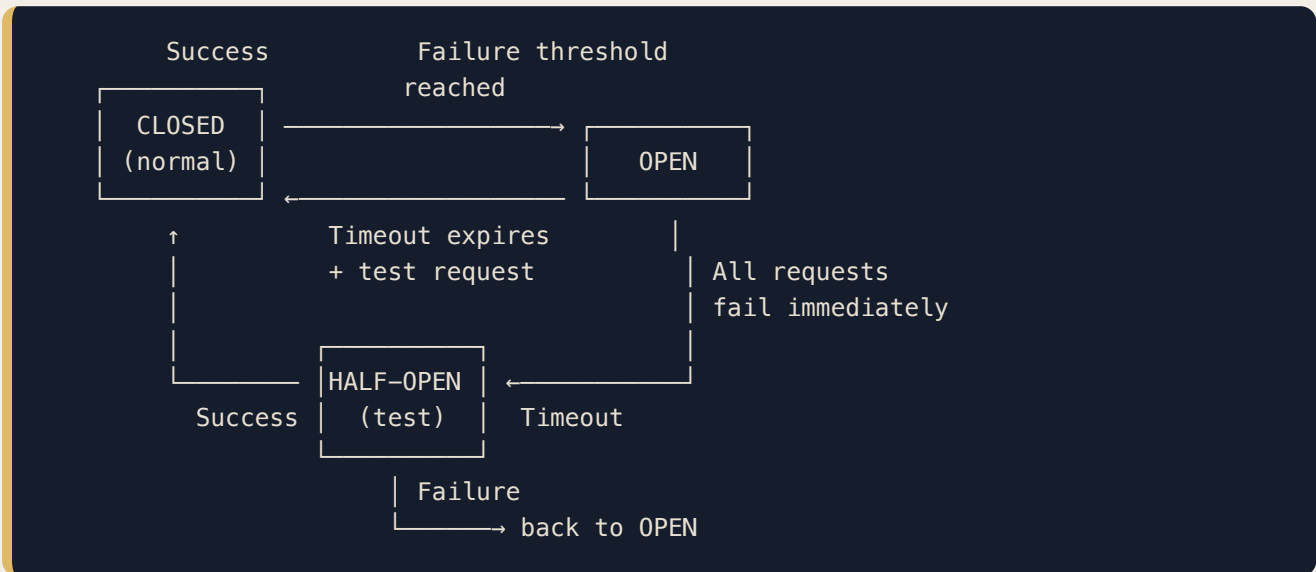
- Mobile BFF: Optimized payloads, fewer round trips
- Web BFF: Full data, pagination
- Admin BFF: Bulk operations, analytics

3. Resilience Patterns

Circuit Breaker

Problem: Calling a failing service repeatedly wastes resources and cascades failures.

States:



Implementation:

```
class CircuitBreaker {
  constructor(options = {}) {
    this.failureThreshold = options.failureThreshold || 5;
    this.resetTimeout = options.resetTimeout || 30000;
    this.state = 'CLOSED';
    this.failureCount = 0;
    this.lastFailureTime = null;
  }

  async execute(fn) {
    if (this.state === 'OPEN') {
      if (Date.now() - this.lastFailureTime >= this.resetTimeout) {
        this.state = 'HALF-OPEN';
      } else {
        throw new Error('Circuit breaker is OPEN');
      }
    }

    try {
      const result = await fn();
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      throw error;
    }
  }

  onSuccess() {
    this.failureCount = 0;
    this.state = 'CLOSED';
  }

  onFailure() {
    this.failureCount++;
    this.lastFailureTime = Date.now();
    if (this.failureCount >= this.failureThreshold) {
      this.state = 'OPEN';
    }
  }
}
```

Libraries: resilience4j (Java), opossum (Node.js), Hystrix (deprecated but concept lives on)

Bulkhead Pattern

Problem: One slow/failing dependency consumes all threads/connections, starving other operations.

Solution: Isolate resources per dependency — like ship bulkheads that contain flooding.

Without Bulkhead:

```
Thread Pool (100)
Service A: 80
Service B: 20
Service A hangs →
ALL 100 blocked
```

With Bulkhead:

Pool A (30)	Pool B (30)
Svc A	Svc B

A hangs → only A's 30 blocked
B continues normally

Implementation:

- Separate thread pools per downstream service
- Separate connection pools per database
- Kubernetes: resource limits per pod/container
- Rate limiting per tenant/service

Retry with Exponential Backoff

```
async function retryWithBackoff(fn, maxRetries = 3, baseDelay = 1000) {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      return await fn();
    } catch (error) {
      if (attempt === maxRetries) throw error;
      if (!isRetryable(error)) throw error;

      const delay = baseDelay * Math.pow(2, attempt) + Math.random() * 1000;
      await sleep(delay);
    }
  }
}

function isRetryable(error) {
  return error.status >= 500 || error.code === 'ECONNRESET';
}
```

Rules:

- Only retry idempotent operations (or with idempotency keys)
- Add jitter to prevent thundering herd
- Set max retries (3-5 typically)
- Combine with circuit breaker (don't retry when circuit is open)

Timeout Pattern

Every external call must have a timeout:

```
async function callWithTimeout(promise, timeoutMs = 5000) {  
  const timeout = new Promise((_, reject) =>  
    setTimeout(() => reject(new Error('Timeout')), timeoutMs)  
  );  
  return Promise.race([promise, timeout]);  
}
```

Cascading timeout budget: If API gateway timeout is 30s and calls 3 services sequentially, each gets ~8s max (with buffer).

4. Apache Kafka Deep Dive

Core Concepts

```
Producer → Topic (Partition 0, 1, 2...) → Consumer Group  
          ↓  
        Broker Cluster  
        (Kafka brokers store  
         and replicate data)
```

Topic: Named stream of records (like a database table or folder) **Partition:** Ordered, immutable sequence of records within a topic **Offset:** Unique sequential ID for each record within a partition **Broker:** Kafka server that stores data and serves clients **Consumer Group:** Set of consumers that jointly consume a topic

Topics & Partitions

Topic: "orders" (3 partitions)

Partition 0: [msg0, msg1, msg2, msg3, msg4] → offsets 0-4

Partition 1: [msg0, msg1, msg2] → offsets 0-2

Partition 2: [msg0, msg1, msg2, msg3] → offsets 0-3

Key-based partitioning:

hash(order_id) % 3 → determines partition

Same key → same partition → ordering guaranteed per key

Why partitions matter:

- **Parallelism:** More partitions = more consumers can read in parallel
- **Ordering:** Guaranteed within a partition, NOT across partitions
- **Scalability:** Partitions distributed across brokers

Partition key strategy:

```
// Good: Orders for same user go to same partition (ordering per user)
const message = {
  key: order.userId,
  value: JSON.stringify(order),
  topic: 'orders',
};
```

```
// Bad: Random keys → no ordering guarantee when needed
```

Consumer Groups

Topic "orders" (4 partitions)

Consumer Group "order-processors":

Consumer 1 → Partition 0, Partition 1

Consumer 2 → Partition 2

Consumer 3 → Partition 3

Rules:

- Each partition consumed by exactly ONE consumer in a group
- Consumers > partitions → idle consumers
- Consumer dies → partitions rebalanced to remaining consumers

Rebalance: When consumers join/leave, partitions are reassigned. During rebalance, consumption pauses (stop-the-world in older versions; cooperative sticky assignor reduces impact).

Offset Management

Auto-commit (default):

```
const consumer = kafka.consumer({
  groupId: 'order-processors',
  autoCommit: true,
  autoCommitInterval: 5000, // commit every 5 seconds
});
// Risk: message processed but crash before commit → reprocessed
// Risk: commit before processing → message lost on crash
```

Manual commit (recommended for critical data):

```
const consumer = kafka.consumer({ groupId: 'order-processors' });

await consumer.run({
  eachMessage: async ({ topic, partition, message }) => {
    await processOrder(JSON.parse(message.value));

    // Commit AFTER successful processing
    await consumer.commitOffsets([
      { topic, partition, offset: (parseInt(message.offset) + 1).toString() },
    ]);
  },
});
```

Offset storage: Kafka stores offsets in internal topic `__consumer_offsets`. Alternative: store offsets in external store (database) for exactly-once with DB writes.

Exactly-Once Semantics (EOS)

Kafka provides three processing guarantees:

Guarantee	Config	Use Case
At-most-once	autoCommit before processing	Metrics, logs
At-least-once	manual commit after processing	Most applications
Exactly-once	transactional + idempotent	Financial processing

Idempotent Producer:

```
const producer = kafka.producer({
  idempotent: true,           // PID + sequence numbers
  maxInFlightRequests: 5,
  retries: Infinity,
});
// Broker deduplicates based on producer ID + sequence number
```

Transactional Producer:

```
const producer = kafka.producer({
  transactionalId: 'order-processor-1',
  idempotent: true,
});

const transaction = await producer.transaction();
try {
  await transaction.send({ topic: 'output', messages: processedOrders });
  await transaction.sendOffsets({
    consumerGroupId: 'order-processors',
    topics: [{ topic: 'orders', partitions: [{ partition: 0, offset: '42' }] }],
  });
  await transaction.commit();
} catch (error) {
  await transaction.abort();
}
```

Kafka Architecture Internals

Replication:

Topic "orders", Partition 0, Replication Factor 3:

Broker 1: [Leader] ← all reads/writes
Broker 2: [Follower] ← replicates from leader
Broker 3: [Follower] ← replicates from leader

ISR (In-Sync Replicas): replicas caught up with leader
min.insync.replicas=2: producer acks=all requires 2 replicas

Log segments:

- Partition log split into segment files (default 1GB)
- Old segments deleted based on retention policy (time or size)
- Compacted topics: keep latest value per key (changelog topics)

ZooKeeper vs KRaft:

- Legacy: Kafka uses ZooKeeper for metadata (broker registration, leader election, config)
- KRaft (Kafka 3.x+): Built-in Raft-based metadata quorum, removes ZooKeeper dependency

Kafka Performance Tuning

Parameter	Default	Tuning
<code>num.partitions</code>	1	Match consumer parallelism (typically 3-12)
<code>replication.factor</code>	1	3 for production
<code>min.insync.replicas</code>	1	2 (with RF=3)
<code>batch.size</code>	16KB	64KB-1MB for throughput
<code>linger.ms</code>	0	5-100ms (batch more messages)
<code>compression.type</code>	none	lz4 or zstd
<code>fetch.min.bytes</code>	1	Increase for throughput

5. Message Queue Comparison

RabbitMQ vs Kafka vs AWS SQS

Feature	RabbitMQ	Apache Kafka	AWS SQS
Model	Message broker (smart broker, dumb consumer)	Distributed commit log (dumb broker, smart consumer)	Managed queue service
Message retention	Deleted after ack	Configurable (days/forever)	14 days max
Ordering	Per queue (single consumer)	Per partition	FIFO queues only
Throughput	~10-50K msg/sec	~1M+ msg/sec	~3000 msg/sec per queue
Consumer model	Push (broker delivers)	Pull (consumer polls)	Pull (long polling)
Replay	No (message deleted on ack)	Yes (reset offset)	No
Routing	Rich (exchanges, bindings, topics)	Topic + partition key	Queue-based
Protocol	AMQP, MQTT, STOMP	Custom binary protocol	HTTP/HTTPS
Ops burden	Self-managed cluster	Self-managed cluster	Fully managed
Best for	Task queues, RPC, complex routing	Event streaming, log aggregation, analytics	Simple decoupling, AWS-native

Decision Matrix

Choose RabbitMQ when:

- Complex routing (topic exchanges, headers exchanges)
- Task distribution with acknowledgment
- Request-reply patterns
- Moderate throughput (< 50K msg/sec)
- Messages should be deleted after processing

Choose Kafka when:

- Event sourcing / event-driven architecture
- Need to replay events
- High throughput (millions of events/sec)
- Stream processing (Kafka Streams, Flink)
- Multiple consumers need same data (pub/sub at scale)
- Log aggregation and analytics pipelines

Choose SQS when:

- Already on AWS, want zero ops
- Simple producer-consumer decoupling
- Variable/unpredictable traffic (auto-scales)
- Don't need message replay or ordering (standard queue)
- Cost optimization for low-volume workloads

Hybrid Architectures

Many production systems use multiple:

```
User Action → API → RabbitMQ (task queue) → Worker processes task
                                     → Kafka (event stream) → Analytics pipeline
                                                         → Audit log
                                                         → Real-time dashboard
                                     → SQS (async job) → Lambda function
```

6. Event-Driven Architecture**Event-Driven vs Request-Driven**

Request-Driven (Synchronous):

```
Client → API Gateway → Order Service → Payment Service → Response
                        (waits)           (waits)
```

- Tight coupling, cascading latency

- Simple to reason about
- Failure in payment = failed order request

Event-Driven (Asynchronous):

```
Client → API Gateway → Order Service → Publish "OrderCreated" → Response (202)
                                     ↓
                               Payment Service (consumes event)
                                     ↓
                               Publish "PaymentProcessed"
                                     ↓
                               Shipping Service (consumes event)
```

- Loose coupling, services don't know about each other
- Eventual consistency
- Failure in payment → retry/DLQ, order already created

Event Types

Type	Description	Example
Event Notification	Minimal info, consumer must query for details	<code>{ type: "OrderCreated", orderId: "123" }</code>
Event-Carried State Transfer	Full state in event, no query needed	<code>{ type: "OrderCreated", order: { id, items, total, user } }</code>
Event Sourcing	Event IS the state, stored in event log	<code>{ type: "ItemAdded", itemId, quantity }</code>
Domain Event	Business-meaningful occurrence	<code>OrderPlaced , PaymentFailed , InventoryReserved</code>

Event Schema Management

Schema Registry (Confluent):

```
// Producer registers schema
const schema = {
  type: 'record',
  name: 'OrderCreated',
  fields: [
    { name: 'orderId', type: 'string' },
    { name: 'total', type: 'double' },
    { name: 'items', type: { type: 'array', items: 'string' } },
  ],
};

// Compatibility modes:
// BACKWARD: new schema can read old data (add optional fields)
// FORWARD: old schema can read new data (remove optional fields)
// FULL: both backward and forward
```

Versioning strategy:

- Add optional fields (backward compatible)
- Never remove required fields
- Use schema registry to enforce compatibility
- Include `schemaVersion` in event metadata

Dead Letter Queue (DLQ)

```
async function processMessage(message) {
  try {
    await handleOrder(JSON.parse(message.value));
    await commitOffset(message);
  } catch (error) {
    message.retryCount = (message.retryCount || 0) + 1;

    if (message.retryCount >= 3) {
      await dlqProducer.send({
        topic: 'orders-dlq',
        messages: [{ key: message.key, value: message.value, headers: { error: error } }],
      });
      await commitOffset(message); // Don't reprocess forever
    } else {
      throw error; // Will be retried
    }
  }
}
```

7. CQRS & Event Sourcing

CQRS (Command Query Responsibility Segregation)

Principle: Separate read and write models.



Why CQRS:

- Read and write patterns differ (reads: complex queries, joins; writes: simple inserts)
- Scale reads independently (add read replicas, caches, search indexes)
- Optimize each model for its purpose
- Different teams can own read vs write paths

Example: E-commerce product catalog

- Write: `CreateProduct` , `UpdatePrice` → PostgreSQL (ACID, normalized)
- Read: `SearchProducts` , `GetProductDetails` → Elasticsearch (full-text search, faceted filtering)
- Sync: `ProductCreated/Updated` events → update Elasticsearch index

When NOT to use CQRS:

- Simple CRUD applications
- Read and write patterns are similar
- Team lacks distributed systems experience
- Eventual consistency on reads is unacceptable

Event Sourcing

Principle: Store state changes as a sequence of events, not current state.

Traditional:

```
Account Balance
$150
```

Update row directly

Event Sourced:

```
Event Store
1. AccountCreated ($0)
2. MoneyDeposited ($200)
3. MoneyWithdrawn ($50)
Current balance = $150
(computed from events)
```

Benefits:

- Complete audit trail (every state change recorded)
- Temporal queries ("what was balance on Jan 1?")
- Replay events to rebuild state or create new projections
- Debug by replaying exact sequence of events

Challenges:

- Event schema evolution (upcasting old events)
- Snapshots needed for performance (replay 1M events is slow)
- Complexity: eventual consistency, CQRS typically paired
- No standard DELETE (append compensating event instead)

Snapshot pattern:

```
Every 100 events, store snapshot:
Snapshot at event 100: { balance: $150, version: 100 }
Replay: load snapshot + events 101-105
Instead of replaying all 105 events
```

CQRS + Event Sourcing Together

```
Command → Aggregate → Generate Events → Event Store
                                     ↓
                               Event Handlers
                               |— Update Read Model
                               |— Send Notifications
                               |— Update Analytics
```

Common in: banking, insurance, order management, collaborative editing.

8. gRPC vs REST vs GraphQL

Comparison Matrix

Feature	REST	gRPC	GraphQL
Protocol	HTTP/1.1 or HTTP/2	HTTP/2	HTTP/1.1 or HTTP/2
Payload	JSON (text)	Protobuf (binary)	JSON (text)
Contract	OpenAPI/Swagger (optional)	.proto files (required)	Schema (required)
Streaming	Limited (SSE, WebSocket)	Bidirectional streaming	Subscriptions
Browser support	Native	Requires grpc-web proxy	Native
Code generation	Optional	Built-in	Built-in
Caching	HTTP caching (CDN-friendly)	Not HTTP-cacheable	POST-only (limited caching)
Performance	Good	Excellent (3-10x faster)	Good (but N+1 risk)
Learning curve	Low	Medium	Medium

REST (Representational State Transfer)

Best for: Public APIs, CRUD operations, browser clients, CDN caching

```
// REST endpoints
GET    /api/users/123      → Get user
POST   /api/users         → Create user
PUT    /api/users/123     → Update user
DELETE /api/users/123     → Delete user
GET    /api/users/123/orders → Get user's orders
```

Strengths: Universal, simple, cacheable, stateless, huge ecosystem **Weaknesses:** Over-fetching (get entire user when you need name), under-fetching (N+1 requests for related data), no streaming

gRPC (Google Remote Procedure Call)

Best for: Internal microservice communication, high-performance, streaming

```
// user.proto
service UserService {
  rpc GetUser (GetUserRequest) returns (User);
  rpc ListUsers (ListUsersRequest) returns (stream User);
  rpc CreateUser (CreateUserRequest) returns (User);
}

message User {
  string id = 1;
  string name = 2;
  string email = 3;
}
```

```
// gRPC client
const client = new UserServiceClient('localhost:50051', credentials.createInsecure);
const user = await client.getUser({ id: '123' });
```

Strengths: Binary serialization (fast), HTTP/2 multiplexing, bidirectional streaming, strong typing via protobuf, code generation **Weaknesses:** Not browser-native, harder to debug (binary), requires .proto management, less human-readable

Streaming types:

- Unary: request → response (like REST)
- Server streaming: request → stream of responses (stock ticker)
- Client streaming: stream of requests → response (file upload)
- Bidirectional: stream ↔ stream (chat)

GraphQL

Best for: Client-driven data fetching, mobile apps (bandwidth-sensitive), rapid frontend iteration

```
# Client specifies exactly what data it needs
query {
  user(id: "123") {
    name
    email
    orders(last: 5) {
      id
      total
      items { name quantity }
    }
  }
}
```

Strengths: No over/under-fetching, single request for complex data, strong typing, introspection

Weaknesses: N+1 query problem (needs DataLoader), caching complexity, query complexity attacks (need depth limiting), not ideal for file uploads or streaming

DataLoader (solves N+1):

```
const userLoader = new DataLoader(async (userIds) => {
  const users = await db.users.findMany({ where: { id: { in: userIds } } });
  return userIds.map(id => users.find(u => u.id === id));
});

// Resolvers batch automatically within same tick
const userResolver = { orders: (user) => orderLoader.load(user.id) };
```

When to Use What at Scale

Scenario	Choice	Why
Public API for third parties	REST	Universal, documented, cacheable
Internal service-to-service	gRPC	Performance, typing, streaming
Mobile/web frontend API	GraphQL	Flexible queries, reduce round trips
Real-time data feed	gRPC streaming or WebSocket	Bidirectional, low latency
File upload/download	REST	Multipart, CDN integration
Admin dashboard (complex queries)	GraphQL	Aggregated data in one request
High-throughput data pipeline	gRPC	Binary, multiplexed

Polyglot approach (common at FAANG):

- External: REST or GraphQL (BFF)
- Internal: gRPC between services
- Events: Kafka for async communication

9. Interview Q&A (15 Questions)

Q1: When would you choose microservices over a monolith?

Answer: I would choose microservices when the organization and product complexity justify the operational overhead. Specifically: when we have 10+ engineers across multiple teams that need to deploy independently; when different components have vastly different scaling requirements (e.g., auth handles 10K QPS while reporting handles 100 QPS); when we need technology diversity (Python for ML pipeline, Go for high-throughput workers, Node.js for API layer); or when fault isolation is critical (a bug in the recommendation engine shouldn't take down checkout).

However, I would NOT start with microservices. I'd begin with a well-structured modular monolith with clear bounded contexts. I'd extract services only when there's a concrete pain point — a module that needs independent scaling, a team that needs deployment autonomy, or a component that

needs a different technology stack. Premature microservices extraction is one of the most common architectural mistakes at startups. The Strangler Fig pattern lets you migrate incrementally, reducing risk.

Q2: Explain the circuit breaker pattern and when you'd use it.

Answer: The circuit breaker pattern prevents cascading failures by stopping calls to a failing downstream service. It operates in three states: Closed (normal operation, requests pass through), Open (failure threshold reached, all requests fail immediately without calling the service), and Half-Open (after a timeout, allow one test request; if it succeeds, return to Closed; if it fails, return to Open).

I'd use circuit breakers for every external dependency — other microservices, third-party APIs, databases. For example, if our payment service starts failing (maybe Stripe is down), without a circuit breaker, our order service would keep calling it, wasting thread pool slots, increasing latency for all users, and potentially causing our entire order service to become unresponsive. With a circuit breaker, after 5 consecutive failures, we stop calling payment service for 30 seconds, return a graceful "payment temporarily unavailable" message, and optionally queue the order for later processing.

Critical implementation details: configure appropriate thresholds (5 failures in 10 seconds), set recovery timeout (30-60 seconds), expose circuit state in metrics (Prometheus), and provide fallback behavior (cached response, default value, or queued retry). Combine with bulkhead pattern to isolate thread pools per dependency.

Q3: How does Kafka ensure message ordering?

Answer: Kafka guarantees message ordering within a partition, but NOT across partitions. This is a fundamental design trade-off — global ordering requires a single partition, which becomes a throughput bottleneck.

Ordering is achieved through partition assignment: messages with the same key always go to the same partition (via `hash(key) % numPartitions`). Within a partition, messages have sequential offsets and are consumed in order. For example, all events for `user_id=123` go to the same partition, so `UserCreated` → `UserUpdated` → `UserDeleted` are processed in order.

To leverage this: choose partition keys based on your ordering requirements. For an order processing system, use `order_id` as the key so all events for an order are ordered. For a user activity feed, use

`user_id` . If you need global ordering (rare), use a single partition — but accept the throughput limit (~10-50MB/sec per partition).

Consumer-side ordering requires: one consumer per partition within a group (automatically handled), processing messages sequentially within each partition (don't parallelize within a partition consumer), and committing offsets only after successful processing. During consumer rebalance, use cooperative sticky assignor to minimize partition movement.

Q4: Explain Kafka consumer groups and rebalancing.

Answer: A consumer group is a set of consumers that jointly consume topics. Kafka assigns each partition to exactly one consumer within a group, enabling parallel processing while ensuring no message is processed twice within the group.

If topic "orders" has 6 partitions and consumer group "processors" has 3 consumers, each consumer gets 2 partitions. If we scale to 6 consumers, each gets 1 partition. If we scale to 8 consumers, 2 sit idle (partitions $\leq$ consumers). Different consumer groups independently consume the same topic — group "analytics" and group "processors" both read all messages.

Rebalancing occurs when consumers join or leave the group (scale up/down, crash, restart). During rebalance, partition ownership is revoked from all consumers, reassigned, and consumption resumes. This causes a brief pause (stop-the-world in older assignors).

To minimize rebalance impact: use `CooperativeStickyAssignor` (incremental rebalance, only move affected partitions), keep `session.timeout.ms` reasonable (45s default), implement rebalance listeners to commit offsets before revocation, and design consumers to be idempotent (safe to reprocess during rebalance). Static membership (`group.instance.id`) prevents rebalance on consumer restart.

Q5: How would you implement exactly-once processing with Kafka?

Answer: Exactly-once processing in Kafka requires coordination between producer, broker, and consumer. True end-to-end exactly-once is achieved through Kafka's transactional API combined with idempotent processing.

Step 1 — Idempotent Producer: Enable with `enable.idempotence=true` . Kafka assigns a Producer ID (PID) and tracks sequence numbers per partition. If the producer retries due to network timeout, the broker deduplicates based on PID + sequence number, preventing duplicate writes.

Step 2 — Transactional Producer: Set `transactional.id` (unique per producer instance). Use `beginTransaction()`, send messages, `sendOffsetsToTransaction()` (atomically commit consumer offsets with produced messages), and `commitTransaction()`. If any step fails, `abortTransaction()` rolls back all writes and offset commits.

Step 3 — Read-Process-Write pattern:

```
const producer = kafka.producer({ transactionalId: 'processor-1', idempotent: true })
await producer.connect();

while (true) {
  const transaction = await producer.transaction();
  try {
    const messages = await consumer.poll();
    const results = await processMessages(messages);
    await transaction.send({ topic: 'output', messages: results });
    await transaction.sendOffsets({ consumerGroupId, topics: offsets });
    await transaction.commit();
  } catch (e) {
    await transaction.abort();
  }
}
```

This guarantees that output messages and offset commits happen atomically — either both succeed or neither does. Combined with idempotent processing logic (dedup table), this achieves effective exactly-once. Use for financial transactions, billing events. For metrics/logs, at-least-once is sufficient and much simpler.

Q6: Compare RabbitMQ, Kafka, and SQS. When would you use each?

Answer: These three serve different messaging paradigms and choosing the wrong one creates significant operational pain.

RabbitMQ is a smart message broker using AMQP. Producers send to exchanges, which route to queues based on bindings (direct, topic, fanout, headers). Consumers receive pushed messages and acknowledge. Messages are deleted after acknowledgment. Best for task queues ("process this email"), request-reply patterns, complex routing logic, and moderate throughput (< 50K msg/sec). Example: order service publishes "SendConfirmationEmail" to a direct exchange routed to the email queue, consumed by email workers.

Kafka is a distributed commit log. Messages are appended to partitioned topics and retained (hours to forever). Consumers pull messages and track their own offsets. Best for event streaming

("something happened"), high throughput (millions/sec), replay capability, multiple independent consumers of the same data, and stream processing. Example: every user action published to Kafka, consumed by recommendation engine, analytics pipeline, and audit log independently.

AWS SQS is a fully managed queue. Simple send/receive with visibility timeout. No message replay, limited ordering (FIFO queues only), 14-day retention max. Best for decoupling components in AWS, async job processing with Lambda, variable traffic (auto-scales), and when you want zero operational overhead. Example: image upload triggers SQS message, Lambda processes resize/compress.

In practice, mature architectures use multiple: Kafka for event backbone, RabbitMQ or SQS for task dispatch, and Kafka Connect to bridge between them.

Q7: What is the Saga pattern and how does it differ from 2PC?

Answer: Saga is a pattern for managing distributed transactions across microservices without requiring a central coordinator to lock resources across services (unlike 2PC).

A saga is a sequence of local transactions, each updating one service's database and publishing an event. If any step fails, compensating transactions undo the preceding steps. For example: CreateOrder → ChargePayment → ReserveInventory → CreateShipment. If ReserveInventory fails, compensate: RefundPayment → CancelOrder.

Key differences from 2PC:

Aspect	2PC	Saga
Consistency	Strong (atomic)	Eventual (no isolation)
Blocking	Yes (locks held)	No (each step commits immediately)
Failure handling	Rollback via coordinator	Compensating transactions
Availability	Low during coordinator failure	High (no central blocker)
Complexity	Simple protocol, hard failure modes	Complex compensating logic
Use case	Single-org, colocated DBs	Microservices, long-running processes

Saga accepts that other transactions may see intermediate states (no isolation). An order might show as "created" before payment is confirmed. This is acceptable for most business processes but

not for operations requiring strict isolation.

Implementation: choreography (event-driven, decentralized) for simple flows; orchestration (central coordinator like Temporal/Step Functions) for complex flows with 5+ steps. Every saga step must be idempotent, and compensating actions must be defined for every step.

Q8: Explain CQRS. When would you implement it?

Answer: CQRS (Command Query Responsibility Segregation) separates the model for writing data (commands) from the model for reading data (queries). Instead of a single database serving both reads and writes, you have optimized write and read stores synchronized via events.

The write side handles commands (CreateOrder, UpdateInventory) with a normalized, ACID-compliant database (PostgreSQL). The read side maintains denormalized views optimized for queries (Elasticsearch for search, Redis for hot data, MongoDB for flexible schemas). When a command succeeds, an event is published, and read model projectors update the read stores.

Implement CQRS when:

1. Read and write patterns diverge significantly (complex searches vs simple writes)
2. Read/write ratio is heavily skewed (100:1 or more)
3. Different scaling requirements (reads need 10x capacity of writes)
4. Multiple read views needed from same write data (admin dashboard, user feed, analytics)
5. Team structure separates read and write ownership

Do NOT implement CQRS when:

- Simple CRUD with similar read/write patterns
- Strong consistency required on reads (CQRS introduces eventual consistency)
- Small team without distributed systems experience
- The added complexity doesn't solve a real problem

Example: E-commerce product catalog. Writes (admin creates/updates products) go to PostgreSQL. Reads (customer search with filters, facets, autocomplete) go to Elasticsearch, updated via ProductCreated/ProductUpdated events. This avoids expensive JOIN queries on every customer search while maintaining ACID writes.

Q9: What is event sourcing and what are its trade-offs?

Answer: Event sourcing stores all changes to application state as a sequence of events, rather than storing current state directly. The current state is derived by replaying events.

Instead of `UPDATE accounts SET balance = 150 WHERE id = 1`, you append events:

`AccountCreated(id=1, balance=0)`, `MoneyDeposited(id=1, amount=200)`, `MoneyWithdrawn(id=1, amount=50)`. Current balance = sum of events = 150.

Benefits:

- Complete audit trail — every state change is permanently recorded with timestamp and context
- Temporal queries — "What was the account balance on March 1?" — replay events up to that date
- Debugging — reproduce exact sequence of events that led to a bug
- Flexibility — create new read projections by replaying events (add analytics view without migrating data)
- Natural fit with event-driven architecture and CQRS

Trade-offs:

- Complexity — event schema evolution requires upcasting; snapshots needed for performance (can't replay 1M events on every read)
- No DELETE — append a compensating event (`AccountClosed`) instead; GDPR "right to be forgotten" requires crypto-shredding
- Eventual consistency on projections — read models lag behind write events
- Learning curve — fundamentally different from CRUD thinking
- Storage grows indefinitely — need compaction/archival strategy

Use event sourcing for domains with strong audit requirements (banking, healthcare), complex state machines (order lifecycle), or collaborative systems (Google Docs-style). Avoid for simple CRUD where the complexity isn't justified.

Q10: Compare gRPC, REST, and GraphQL for microservices communication.

Answer: Each protocol serves different needs in a microservices architecture, and mature systems typically use all three in different layers.

REST is the universal choice for external/public APIs. JSON over HTTP, cacheable, well-understood, huge tooling ecosystem. Weaknesses: over-fetching (GET /users/123 returns all fields when you need name only), under-fetching (need 5 requests to assemble a page), no native streaming. Use for third-party integrations, browser-facing APIs, and when CDN caching matters.

gRPC is the internal workhorse. Protobuf binary serialization is 3-10x faster than JSON. HTTP/2 enables multiplexing (multiple requests on one connection). Built-in code generation from .proto files ensures type safety. Supports unary, server streaming, client streaming, and bidirectional streaming. Weaknesses: not browser-native (needs grpc-web proxy), harder to debug (binary), requires proto file management. Use for service-to-service communication, high-throughput pipelines, and real-time streaming (chat, live updates).

GraphQL excels at client-driven data fetching. Clients specify exactly what fields they need in one request, eliminating over/under-fetching. Strong typing via schema, introspection for tooling. Weaknesses: N+1 query problem (solve with DataLoader), POST-only (limited HTTP caching), query complexity attacks (need depth/cost limiting). Use for mobile apps (bandwidth-sensitive), complex UIs needing aggregated data, and when frontend teams want independence from backend API design.

Recommended architecture: GraphQL/REST BFF (Backend for Frontend) as the external API layer → gRPC for internal service communication → Kafka for async events. This gives you the best of each: developer-friendly external API, high-performance internal communication, and reliable async processing.

Q11: What is the outbox pattern and why is it critical in event-driven microservices?

Answer: The outbox pattern solves the fundamental dual-write problem in event-driven microservices: you need to atomically update your database AND publish an event, but these are two separate systems that can fail independently.

Without outbox: `db.update(order)` succeeds, then `kafka.publish('OrderCreated')` fails → database says order exists but no downstream service knows about it. Or Kafka succeeds but DB fails → ghost event with no corresponding data.

With outbox: within a single database transaction, update the business table AND insert a row into an outbox table. A separate relay process (polling or CDC via Debezium) reads unpublished outbox entries, publishes to Kafka, and marks them as published.

This guarantees at-least-once event delivery (the event will eventually be published). Combined with idempotent consumers, you get effective exactly-once processing.

Why critical: in microservices, events are the contract between services. A missed event means payment service never charges, inventory never reserves, or shipping never triggers. The outbox pattern is the industry-standard solution used by Uber (Cadence), Netflix, and virtually every mature event-driven architecture. It's non-negotiable for any system where event reliability matters.

Implementation tip: use CDC (Debezium) over polling for lower latency and less database load. The outbox table becomes a Kafka Connect source, reading the WAL/binlog directly.

Q12: How would you handle a poison message in a Kafka consumer?

Answer: A poison message is one that consistently causes processing failure — malformed data, references a deleted entity, triggers a bug in consumer code. Without handling, it blocks the partition (manual commit won't advance offset) or causes infinite retry loops.

My handling strategy:

1. **Retry with backoff:** First 3 attempts with exponential backoff (1s, 4s, 16s). Transient failures (network timeout, DB connection) often resolve.
2. **Dead Letter Topic (DLT):** After max retries, publish the message to `orders-dlq` topic with metadata (original topic, partition, offset, error message, retry count). Commit the offset on the original partition so processing continues.
3. **Alerting:** Monitor DLT message rate. Any message in DLT triggers a PagerDuty alert for investigation.
4. **DLT consumer/replay tool:** Build an admin tool to inspect DLT messages, fix the root cause (code bug, data issue), and replay messages back to the original topic.
5. **Schema validation:** Validate messages against schema at consumption time (Schema Registry). Invalid schema → DLT immediately (no retry — won't fix itself).

```
async function processWithDLT(message, attempt = 0) {
  try {
    await processOrder(JSON.parse(message.value));
  } catch (error) {
    if (attempt < 3 && isTransient(error)) {
      await sleep(Math.pow(2, attempt) * 1000);
      return processWithDLT(message, attempt + 1);
    }
    await dlqProducer.send({
      topic: 'orders-dlq',
      messages: [{
        key: message.key,
        value: message.value,
        headers: {
          'original-topic': message.topic,
          'error': error.message,
          'attempts': attempt.toString(),
        },
      }],
    });
  }
}
```

Prevention: idempotent consumers, input validation, circuit breaker on downstream calls, and comprehensive integration tests with edge-case messages.

Q13: Explain the bulkhead pattern with a real-world example.

Answer: The bulkhead pattern isolates components so that failure in one doesn't consume all resources and starve others — named after ship bulkheads that contain flooding to one compartment.

Real-world example: Our e-commerce platform's API server has a thread pool of 200 threads. It calls three downstream services: Product Catalog (fast, 50ms p99), Payment Gateway (moderate, 200ms p99), and Recommendation Engine (slow, 2s p99).

Without bulkheads: Recommendation Engine has an outage and all 200 threads block waiting for timeout (30s). Product Catalog and Payment Gateway calls also fail — the entire site is down because of one slow dependency.

With bulkheads: allocate separate thread pools — Product Catalog: 80 threads, Payment: 80 threads, Recommendations: 40 threads. When Recommendation Engine goes down, only its 40 threads are

affected. Product browsing and checkout continue normally. Users see "Recommendations unavailable" instead of a total outage.

Implementation levels:

- **Thread pool isolation:** Separate pools per downstream service (Hystrix/resilience4j style)
- **Connection pool isolation:** Separate DB connection pools per service/schema
- **Kubernetes:** Resource limits and requests per pod; separate deployments for critical vs non-critical paths
- **Network:** Service mesh (Istio) with per-service circuit breakers and rate limits

Combined with circuit breakers (fail fast when bulkhead is full) and graceful degradation (serve cached recommendations or hide the section), bulkheads ensure resilience. At 80 LPA level, discuss how you'd implement this in your specific stack — thread pools in Node.js (worker threads), connection pools in pg/mysql, or service mesh policies in Kubernetes.

Q14: How would you design an event-driven order processing system?

Answer: I'd design an event-driven order system using choreography-based saga with Kafka as the event backbone and outbox pattern for reliability.

Services: Order Service, Payment Service, Inventory Service, Shipping Service, Notification Service.

Flow:

1. Client POST /orders → Order Service validates, creates order (status: PENDING), writes to DB + outbox table in one transaction
2. Outbox relay publishes `OrderCreated` to Kafka topic `orders.events`
3. Payment Service consumes `OrderCreated`, charges card via Stripe, publishes `PaymentProcessed` or `PaymentFailed`
4. Inventory Service consumes `PaymentProcessed`, reserves items, publishes `InventoryReserved` or `InventoryInsufficient`
5. Shipping Service consumes `InventoryReserved`, creates shipment, publishes `ShipmentCreated`
6. Order Service consumes `ShipmentCreated`, updates status to CONFIRMED
7. Notification Service consumes all events, sends emails at each stage

Failure handling (compensating events):

- `PaymentFailed` → Order Service sets status CANCELLED
- `InventoryInsufficient` → Payment Service publishes `RefundInitiated` → Order Service sets CANCELLED

- Any service crash → Kafka redelivers uncommitted events → idempotent reprocessing

Key design decisions:

- Outbox pattern in every service for reliable event publishing
- Idempotency keys on all event handlers (dedup by event ID)
- Dead letter topics for poison messages
- Schema Registry with backward-compatible Avro schemas
- Order status as state machine (PENDING → PAID → RESERVED → SHIPPED → DELIVERED)
- CQRS for order history queries (Elasticsearch read model from events)
- Monitoring: consumer lag alerts, saga completion time metrics, DLQ rate

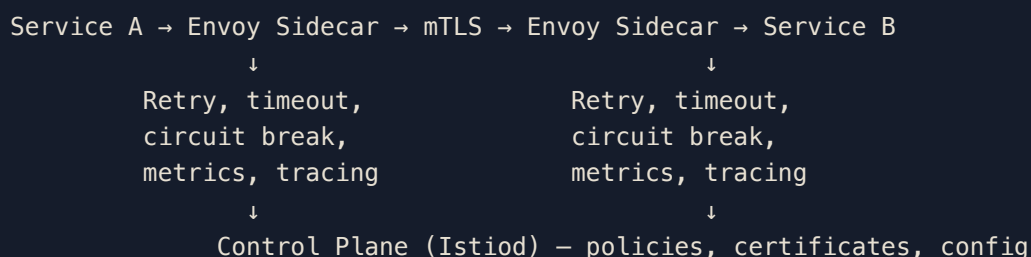
This design gives independent service deployment, fault isolation, and natural audit trail via Kafka event log retention.

Q15: What is a service mesh and when do you need one?

Answer: A service mesh is a dedicated infrastructure layer for handling service-to-service communication. It extracts cross-cutting concerns (networking, security, observability) from application code into sidecar proxies (typically Envoy) deployed alongside each service instance.

Without service mesh: Each service implements retries, timeouts, circuit breakers, mTLS, and tracing individually. Inconsistent implementations, duplicated code, and services written before resilience patterns were adopted become liabilities.

With service mesh (Istio/Linkerd):



What service mesh provides:

- **Traffic management:** Load balancing, retries, timeouts, circuit breaking, canary deployments, traffic mirroring
- **Security:** Automatic mTLS between all services, authorization policies

- **Observability:** Distributed tracing (Jaeger), metrics (Prometheus), access logs — all without code changes

When you NEED a service mesh:

- 20+ microservices with complex inter-service communication
- Strict mTLS requirements (compliance, zero-trust)
- Advanced traffic management (canary, A/B testing at infrastructure level)
- Organization lacks consistency in implementing resilience patterns

When you DON'T need it:

- < 10 services (API gateway + library-based resilience suffices)
- Team lacks Kubernetes/mesh operational expertise
- Performance overhead matters (sidecar adds ~1-3ms latency per hop)
- Simple request-response patterns without complex routing

For most teams scaling to 80 LPA interviews, demonstrate knowledge of service mesh concepts but recommend starting with API gateway + client-side resilience libraries, adopting a mesh only when operational complexity of many services demands centralized control.

10. Where to Practice

Hands-On Labs

- [Confluent Kafka Tutorials](#) — Free Kafka hands-on
- [Kafka Docker Quickstart](#) — Local cluster setup
- [Microservices.io](#) — Chris Richardson's patterns catalog
- [GitHub: node-microservices-demo](#) — Google microservices demo

Books

- "Building Microservices" (2nd Ed) by Sam Newman — Architecture patterns
- "Microservices Patterns" by Chris Richardson — Saga, CQRS, event sourcing
- "Kafka: The Definitive Guide" by Gwen Shapira — Kafka internals
- "Enterprise Integration Patterns" by Hohpe & Woolf — Messaging patterns

Courses

- [Confluent Kafka Fundamentals](#) — Free Kafka course
- [Udemy: Microservices with Node JS and React](#) — Practical microservices
- [ByteByteGo: System Design](#) — Microservices & messaging chapters

Interview Prep

- [Exponent System Design](#) — Microservices design questions
- [Hello Interview](#) — Structured system design
- Design questions: "Design a food delivery system", "Design an notification system", "Design a payment processing system"

Key Topics to Drill

1. Draw microservices architecture with API gateway, Kafka, and 3 services
2. Explain Kafka partition strategy for an ordering use case
3. Walk through Saga pattern for a booking system (flight + hotel + payment)
4. Compare when to use sync (REST/gRPC) vs async (Kafka) communication
5. Design exactly-once processing for a billing pipeline

Last updated: August 2026 | Target: 80+ LPA SDE (Google L5, Meta E4, Amazon SDE3, Microsoft L63+)

Microservices, Kafka, and Event-Driven Architecture Interview – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers microservices architecture, Apache Kafka, message queues, and event-driven design for product company interviews. Spell out abbreviations on first use.

1. What is a microservices architecture?

Microservices architecture splits an application into small, independently deployable services, each owning its data and business logic. Services communicate over the network via HTTP/REST (Representational State Transfer) Application Programming Interfaces (APIs) or message queues. Benefits include independent scaling, team autonomy, and technology diversity. Costs include operational complexity, network latency, and distributed debugging challenges.

2. What is a monolith vs microservices?

A monolith is a single deployable unit containing all features—simpler to develop, test, and deploy initially. Microservices split features into separate services with independent databases and deployment cycles. Start with a modular monolith; extract microservices when teams scale, components need independent scaling, or deployment conflicts arise. Interview tip: justify your choice with team size and scaling needs.

3. What is service discovery?

Service discovery lets services find each other's network locations dynamically without hardcoded URLs. A registry (Consul, Eureka, Kubernetes DNS) tracks healthy instances. Client-side discovery: client queries registry and load-balances. Server-side discovery: load balancer queries registry. Essential in microservices where instances scale up and down constantly.

4. What is an API Gateway in microservices?

An API Gateway is a single entry point for external clients routing requests to internal microservices. Handles cross-cutting concerns: authentication, rate limiting, SSL (Secure Sockets

Layer) termination, request routing, response aggregation, and protocol translation. Examples: Kong, AWS (Amazon Web Services) API Gateway, Nginx. Prevents clients from needing to know internal service topology.

5. What is a message queue?

A message queue is middleware that decouples producers (senders) from consumers (receivers) by buffering messages. Producers publish messages; consumers pull or receive pushed messages asynchronously. Benefits: load leveling, fault tolerance (messages persist if consumer is down), and independent scaling. Examples: Apache Kafka, RabbitMQ, Amazon Simple Queue Service (SQS).

6. What is Apache Kafka?

Apache Kafka is a distributed event streaming platform designed for high-throughput, fault-tolerant, real-time data pipelines. Messages are stored in append-only logs (topics) partitioned across brokers. Producers write; consumers read at their own pace. Used for event sourcing, log aggregation, stream processing, and microservice communication. Handles millions of messages per second.

7. What is a Kafka topic?

A Kafka topic is a named category or feed where messages (records) are published. Topics are divided into partitions for parallelism. Example topics: `order-events`, `user-signups`, `payment-completed`. Producers write to topics; consumers subscribe to topics. Topics are configured with retention period, replication factor, and partition count.

8. What is a Kafka partition?

A Kafka partition is an ordered, immutable sequence of messages within a topic. Partitions enable parallelism—multiple consumers in a group each read different partitions. Messages within a partition are strictly ordered; ordering across partitions is not guaranteed. Choose partition key (e.g., user ID) to keep related messages in the same partition for ordering.

9. What is a Kafka broker?

A Kafka broker is a server that stores topic partitions and serves producer/consumer requests. A Kafka cluster consists of multiple brokers for fault tolerance. Each partition has one leader broker

(handles reads/writes) and zero or more follower brokers (replicate data). Brokers coordinate via Apache ZooKeeper (legacy) or KRaft (Kafka Raft) for metadata management.

10. What is a Kafka producer?

A Kafka producer is a client application that publishes messages to Kafka topics. Producers choose which partition to write to (by key hash or round-robin). They can configure acknowledgment level (acks=0 fire-and-forget, acks=1 leader only, acks=all all in-sync replicas). Idempotent producers prevent duplicate messages on retry using sequence numbers.

11. What is a Kafka consumer?

A Kafka consumer is a client application that reads messages from Kafka topics. Consumers track their read position via offsets. They can read from the beginning, latest, or a specific offset. Consumers subscribe to one or more topics and process messages at their own pace—decoupling processing speed from production speed.

12. What is a consumer group in Kafka?

A consumer group is a set of consumers that jointly consume a topic, with each partition assigned to exactly one consumer in the group. Enables horizontal scaling: add consumers to handle more partitions. If consumers exceed partitions, extras sit idle. Rebalance occurs when consumers join or leave. Different consumer groups independently read the same topic.

13. What is Kafka offset?

An offset is a sequential identifier (position) of a message within a partition. Consumers commit offsets to track progress—on restart, they resume from the last committed offset. Offsets are stored in an internal Kafka topic (`__consumer_offsets`) or externally (database). Uncommitted offsets mean reprocessing on failure (at-least-once semantics).

14. What is auto-commit vs manual commit for offsets?

Auto-commit periodically commits offsets in the background—simple but may commit before processing completes (message loss on crash) or after failure (duplicate processing). Manual commit lets you commit only after successful processing—enables at-least-once or effectively

exactly-once with idempotent handlers. Prefer manual commit for critical business logic like payments.

15. What is Kafka replication factor?

Replication factor is the number of copies of each partition across brokers. Factor 3 means each partition has 1 leader + 2 followers. Higher replication improves fault tolerance but uses more storage. If a broker dies, a follower is promoted to leader. Set replication factor ≥ 3 in production for availability.

16. What is in-sync replicas (ISR) in Kafka?

In-sync replicas (ISR) are follower replicas that are fully caught up with the leader. Only ISR members can become leader on failover. If a follower falls too far behind, it is removed from ISR until it catches up. Producers with `acks=all` wait for all ISR members to acknowledge—balances durability vs latency.

17. What is Kafka retention policy?

Retention policy controls how long messages are kept. Time-based: delete after N days (default 7 days). Size-based: delete when partition exceeds size limit. Log compaction: keep only the latest message per key (for changelog topics). Configure per topic based on use case—event sourcing needs longer retention; transient notifications need shorter.

18. What is log compaction in Kafka?

Log compaction retains only the latest value for each key in a topic, deleting older versions. Useful for changelog topics where only the current state matters (user profile updates, database Change Data Capture). Combined with time retention: compacted segments kept forever (per key), non-compacted data deleted after retention period.

19. What is event-driven architecture (EDA)?

Event-driven architecture (EDA) is a design pattern where services communicate by producing and consuming events (notifications of state changes) rather than direct synchronous calls. A service publishes an event when something happens; interested services react asynchronously. Benefits:

loose coupling, scalability, and resilience. Foundation for microservices at companies like Netflix, Uber, and LinkedIn.

20. What is an event vs a command vs a query?

An event (past tense: `OrderPlaced`) announces something that already happened—multiple subscribers react. A command (imperative: `PlaceOrder`) requests an action—one handler executes it. A query (question: `GetOrderStatus`) requests data—read-only, no side effects. Command Query Responsibility Segregation (CQRS) separates commands and queries. Events enable choreography; commands enable orchestration.

21. What is event sourcing?

Event sourcing stores state changes as a sequence of events rather than overwriting current state. Current state is rebuilt by replaying events. Example: bank account balance computed from all deposit/withdrawal events. Benefits: complete audit trail, temporal queries (state at any point), and easy event replay. Often paired with Apache Kafka as the event store.

22. What is Change Data Capture (CDC)?

Change Data Capture (CDC) tracks and streams database changes (inserts, updates, deletes) to other systems in real time. Tools: Debezium (reads database transaction logs), AWS Database Migration Service. Use cases: sync read replicas, feed search indexes, trigger microservice events. CDC turns a database into an event source without modifying application code.

23. What is RabbitMQ vs Kafka?

RabbitMQ is a traditional message broker—messages deleted after consumption, push-based delivery, flexible routing (exchanges, queues). Good for task queues and request-reply patterns. Kafka is a distributed log—messages retained for configurable period, pull-based consumption, high throughput. Good for event streaming, log aggregation, and replay. Choose RabbitMQ for job queues; Kafka for event streams.

24. What is Amazon SQS vs Kafka?

Amazon Simple Queue Service (SQS) is a fully managed queue service—messages deleted after processing, no replay, simpler operations. Kafka is self-managed (or Amazon Managed Streaming for

Apache Kafka—MSK) with log retention, replay, and stream processing. SQS suits simple async tasks; Kafka suits event-driven architectures needing replay, ordering, and high throughput.

25. What is a dead letter queue (DLQ)?

A dead letter queue (DLQ) stores messages that failed processing after maximum retry attempts. Prevents poison messages (malformed or causing crashes) from blocking the main queue. Operators inspect DLQ to diagnose issues, fix bugs, and replay messages. Configure in Kafka (dead letter topic), RabbitMQ, and Amazon SQS. Essential for production reliability.

26. What is the outbox pattern?

The outbox pattern ensures reliable event publishing alongside database writes. Instead of publishing to Kafka directly (which can fail after DB commit), write the event to an outbox table in the same database transaction. A separate relay process reads the outbox and publishes to Kafka, marking entries as sent. Guarantees at-least-once event delivery without dual-write problems.

27. What is the inbox pattern?

The inbox pattern ensures idempotent message processing in microservices. Store incoming message IDs in an inbox table; before processing, check if the ID already exists—if yes, skip (duplicate). Process the message and record the ID atomically in one transaction. Prevents duplicate side effects from at-least-once message delivery. Pair with the outbox pattern for end-to-end reliability.

28. What is Kafka Connect?

Kafka Connect is a framework for streaming data between Apache Kafka and external systems (databases, object storage, search engines). Source connectors ingest data into Kafka; sink connectors export Kafka data to external systems. Pre-built connectors for PostgreSQL, MySQL, Elasticsearch, Amazon S3. Runs as a distributed, scalable service. Simplifies integration without custom producer/consumer code.

29. What is Kafka Streams?

Kafka Streams is a Java library for building real-time stream processing applications on top of Kafka. Processes messages as they arrive—filtering, aggregating, joining streams, windowing. Stateful

processing with local RocksDB storage and changelog topics for fault tolerance. Alternative to Apache Flink or Apache Spark Streaming for Kafka-native stream processing.

30. What is ksqlDB?

ksqlDB is a streaming SQL engine for Apache Kafka. Write SQL queries to filter, transform, and aggregate Kafka topic streams in real time. Creates new topics from query results. Lowers barrier for stream processing—analysts and backend engineers use familiar SQL syntax. Built on Kafka Streams under the hood. Good for real-time dashboards and alerting.

31. What is schema registry?

Schema registry (Confluent Schema Registry) stores and validates schemas (Avro, Protobuf, JSON Schema) for Kafka message formats. Producers register schemas; consumers fetch schemas to deserialize. Enforces compatibility rules (backward, forward, full) so schema evolution does not break consumers. Prevents unstructured JSON chaos in large Kafka deployments.

32. What is Avro vs JSON for Kafka messages?

JSON (JavaScript Object Notation) is human-readable but verbose and lacks schema enforcement—schema drift breaks consumers silently. Avro is a compact binary format with embedded schema IDs and strong schema evolution support via schema registry. Avro reduces message size 30-50% and prevents incompatible changes. Use Avro or Protobuf in production; JSON for prototyping.

33. What is backpressure in message queues?

Backpressure occurs when consumers cannot keep up with producers, causing queue depth to grow unboundedly. Without backpressure, memory exhaustion and message loss occur. Solutions: bounded queues with rejection, consumer rate limiting, Kafka consumer pause/resume, and monitoring consumer lag. Design consumers to scale horizontally with partition count.

34. What is consumer lag in Kafka?

Consumer lag is the difference between the latest message offset in a partition and the consumer's committed offset—how far behind the consumer is. High lag indicates slow processing or insufficient consumers. Monitor lag per partition; alert when lag exceeds threshold. Scale consumers (up to partition count) or optimize processing to reduce lag.

35. What is a rebalance in Kafka?

Rebalance is the process of redistributing partition assignments among consumer group members when consumers join, leave, or crash. During rebalance, consumption pauses (stop-the-world). Frequent rebalances hurt throughput. Mitigate with static membership, cooperative rebalancing (incremental assignors), and stable consumer count matching partition count.

36. What is the strangler fig pattern?

The strangler fig pattern gradually replaces a monolith by building new microservices around it, routing increasing traffic to new services until the monolith can be retired. Named after strangler fig vines that gradually envelop a tree. Reduces big-bang migration risk. Use an API Gateway to route traffic: old endpoints to monolith, new endpoints to microservices.

37. What is the backend for frontend (BFF) pattern?

Backend for Frontend (BFF) creates separate API layers tailored to each client type (web, mobile, admin). Each BFF aggregates calls to internal microservices and shapes responses for its client. Prevents a single generic API from being too coarse for mobile or too verbose for web. Netflix and Spotify use BFF patterns extensively.

38. What is synchronous vs asynchronous communication in microservices?

Synchronous (HTTP/gRPC—Google Remote Procedure Call): caller waits for response—simple but creates coupling and cascade failures. Asynchronous (message queue): caller publishes event and continues—decoupled but harder to debug. Use sync for queries needing immediate answers; async for events, notifications, and long-running processes. Most systems combine both.

39. What is gRPC and when to use it over REST?

gRPC (Google Remote Procedure Call) uses Protocol Buffers for compact binary serialization and HTTP/2 for multiplexed connections. Supports streaming (client, server, bidirectional). Faster and more efficient than REST/JSON for service-to-service communication. Use gRPC for internal microservice calls; REST for public APIs where browser compatibility and human readability matter.

40. What is a circuit breaker in microservices?

A circuit breaker stops calling a failing downstream microservice after a threshold of failures, returning a fallback response immediately. Prevents cascade failures where one slow service brings down the entire system. States: Closed (normal), Open (fail fast), Half-Open (test recovery). Implement with Resilience4j, Istio service mesh, or API Gateway policies.

41. What is service mesh?

A service mesh is infrastructure layer handling service-to-service communication: load balancing, mutual TLS (Transport Layer Security) encryption, retries, circuit breaking, and observability. Sidecar proxies (Envoy) deployed alongside each service intercept traffic. Control plane (Istio, Linkerd) configures policies. Offloads cross-cutting concerns from application code. Useful when you have 20+ microservices.

42. What is domain-driven design (DDD) in microservices?

Domain-Driven Design (DDD) models software around business domains using bounded contexts—each microservice maps to one bounded context with its own ubiquitous language, data model, and team. Example: Order context, Payment context, Shipping context. Prevents shared databases and unclear service boundaries. Eric Evans' book is the foundational reference.

43. What is the saga pattern with Kafka?

Implement sagas using Kafka events: each step publishes a success event that triggers the next step; failure events trigger compensating actions. Orchestration: a saga manager consumes events and publishes commands. Choreography: services react to each other's events directly. Kafka's durability and replay make it ideal for saga event logs. Use dead letter topics for failed saga steps.

44. What is CQRS (Command Query Responsibility Segregation)?

Command Query Responsibility Segregation (CQRS) separates write operations (commands that change state) from read operations (queries that return data). Write model optimized for consistency; read model optimized for query patterns (denormalized views). Models sync via events. Useful when read and write patterns differ significantly—e.g., write to PostgreSQL, read from Elasticsearch.

45. What is eventual consistency in microservices?

Eventual consistency means that after a write in one microservice, other services' read models will converge to the updated state over time—not instantly. Acceptable for search indexes, analytics, and notification systems. Unacceptable for payment balances without careful design. Mitigate with user-facing indicators ("Processing..."), version numbers, and read-your-writes routing.

46. What is idempotent consumer design?

Idempotent consumers produce the same result whether a message is processed once or multiple times. Implement by storing processed message IDs (inbox pattern), using natural idempotency (setting status to SHIPPED twice is harmless), or conditional updates (UPDATE WHERE status = 'PENDING'). Required because Kafka guarantees at-least-once delivery with retries.

47. What is Kafka security?

Kafka security has three layers: Authentication (who are you?)—SSL certificates, SASL (Simple Authentication and Security Layer) mechanisms. Authorization (what can you do?)—Access Control Lists (ACLs) per topic. Encryption—TLS for data in transit, optional at-rest encryption. Enable all three in production. Amazon MSK integrates with AWS IAM (Identity and Access Management).

48. What is MSK (Managed Streaming for Kafka)?

Amazon Managed Streaming for Apache Kafka (MSK) is AWS's fully managed Kafka service. AWS handles broker provisioning, patching, and replacement. Supports standard Kafka APIs—migrate existing applications without code changes. Integrates with AWS VPC (Virtual Private Cloud), IAM, CloudWatch monitoring, and MSK Connect. Reduces operational burden vs self-managed Kafka clusters.

49. How do you monitor Kafka in production?

Monitor: consumer lag (per group, per partition), broker disk usage, under-replicated partitions, request rate and latency, active controller count, and offline partitions. Tools: Kafka Manager, Confluent Control Center, Prometheus + Grafana, AWS CloudWatch for MSK. Alert on lag spikes, broker failures, and ISR shrinkage. Run synthetic produce/consume health checks.

50. Summarize Kafka and microservices interview checklist?

Cover: why microservices (team scale, independent deploy), communication patterns (sync vs async), Kafka fundamentals (topic, partition, consumer group, offset), reliability patterns (outbox, inbox, idempotent consumer, DLQ), saga for distributed transactions, observability (consumer lag, tracing), and when NOT to use microservices (small team, early product). Draw architecture diagrams and discuss trade-offs confidently.